In [139…

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb
from scipy.stats import pearsonr
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.metrics import r2_score, accuracy_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.inspection import permutation_importance
from sklearn.ensemble import RandomForestRegressor
import xgboost as xgb

df = pd.read_csv("apple_stock_enriched_phase2_output.csv")
df.head()
```

Out[139…

	Title	Date	Link	Source	Open	High	Low	Close	Volur
0	iPhone - Apple Newsroom	2007-01-09 23:23:21+00:00	https://news.google.com/rss/articles/CBMiQ0FVX...	Google RSS	2.601592	2.798103	2.562470	2.785764	33492984
1	Apple Events - Apple Newsroom	2010-07-04 21:57:14+00:00	https://news.google.com/rss/articles/CBMiS0FVX...	Google RSS	7.538144	7.551386	7.318762	7.431313	6938428
2	MacBook Pro - Apple Newsroom	2012-06-11 19:58:27+00:00	https://news.google.com/rss/articles/CBMiSkFVX...	Google RSS	17.686608	17.710081	17.172309	17.188560	5912648
3	Mac mini - Apple Newsroom	2012-10-24 14:03:37+00:00	https://news.google.com/rss/articles/CBMiRkFVX...	Google RSS	18.781662	18.936100	18.455255	18.642334	5585272
4	Inclusion & Diversity - Apple Newsroom	2014-08-12 20:58:05+00:00	https://news.google.com/rss/articles/CBMiR0FVX...	Google RSS	21.271840	21.457891	21.176600	21.256336	1351800

5 rows × 32 columns

In [140…

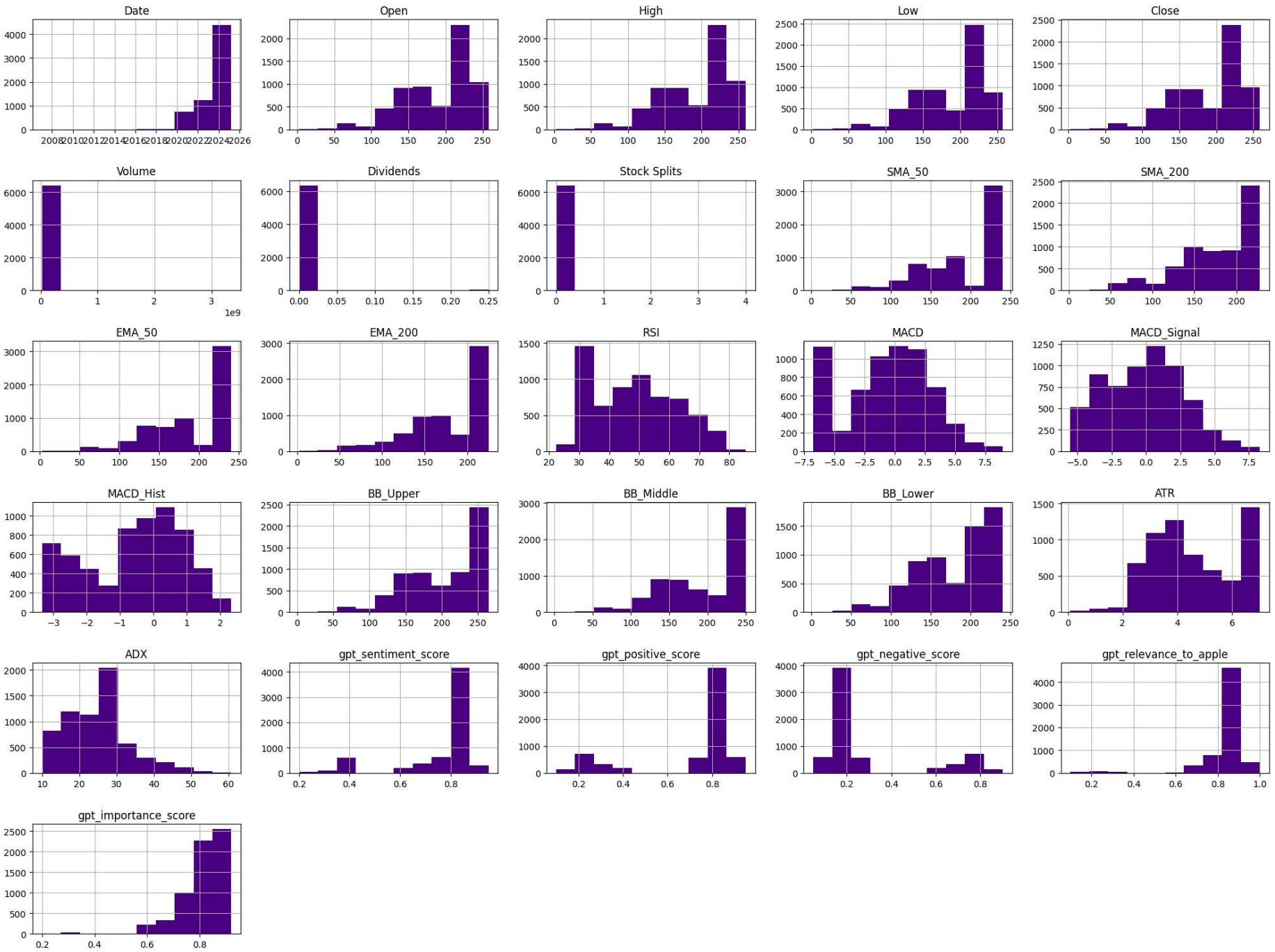
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6415 entries, 0 to 6414
Data columns (total 32 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Title                    6415 non-null   object
 1   Date                     6415 non-null   object
 2   Link                     6415 non-null   object
 3   Source                   6415 non-null   object
 4   Open                     6415 non-null   float64
 5   High                     6415 non-null   float64
 6   Low                      6415 non-null   float64
 7   Close                    6415 non-null   float64
 8   Volume                   6415 non-null   int64
 9   Dividends                6415 non-null   float64
 10  Stock Splits             6415 non-null   float64
 11  SMA_50                   6415 non-null   float64
 12  SMA_200                  6415 non-null   float64
 13  EMA_50                   6415 non-null   float64
 14  EMA_200                  6415 non-null   float64
 15  RSI                      6415 non-null   float64
 16  MACD                     6415 non-null   float64
 17  MACD_Signal              6415 non-null   float64
 18  MACD_Hist                6415 non-null   float64
 19  BB_Upper                 6415 non-null   float64
 20  BB_Middle                6415 non-null   float64
 21  BB_Lower                 6415 non-null   float64
 22  ATR                      6415 non-null   float64
 23  ADX                      6415 non-null   float64
 24  gpt_summary              6415 non-null   object
 25  gpt_sentiment_direction  6415 non-null   object
 26  gpt_sentiment_score      6415 non-null   float64
 27  gpt_positive_score       6415 non-null   float64
 28  gpt_negative_score       6415 non-null   float64
 29  gpt_event_type           6415 non-null   object
 30  gpt_relevance_to_apple   6415 non-null   float64
 31  gpt_importance_score     6415 non-null   float64
dtypes: float64(24), int64(1), object(7)
memory usage: 1.6+ MB
```

In [141…
```
df["Date"] = pd.to_datetime(df["Date"])
```

In [142…
```
df = df.sort_values("Date").reset_index(drop=True)
```

In [143…
```
df.hist(figsize=(20,15), bins=10, color='indigo')
plt.tight_layout()
plt.show()
```

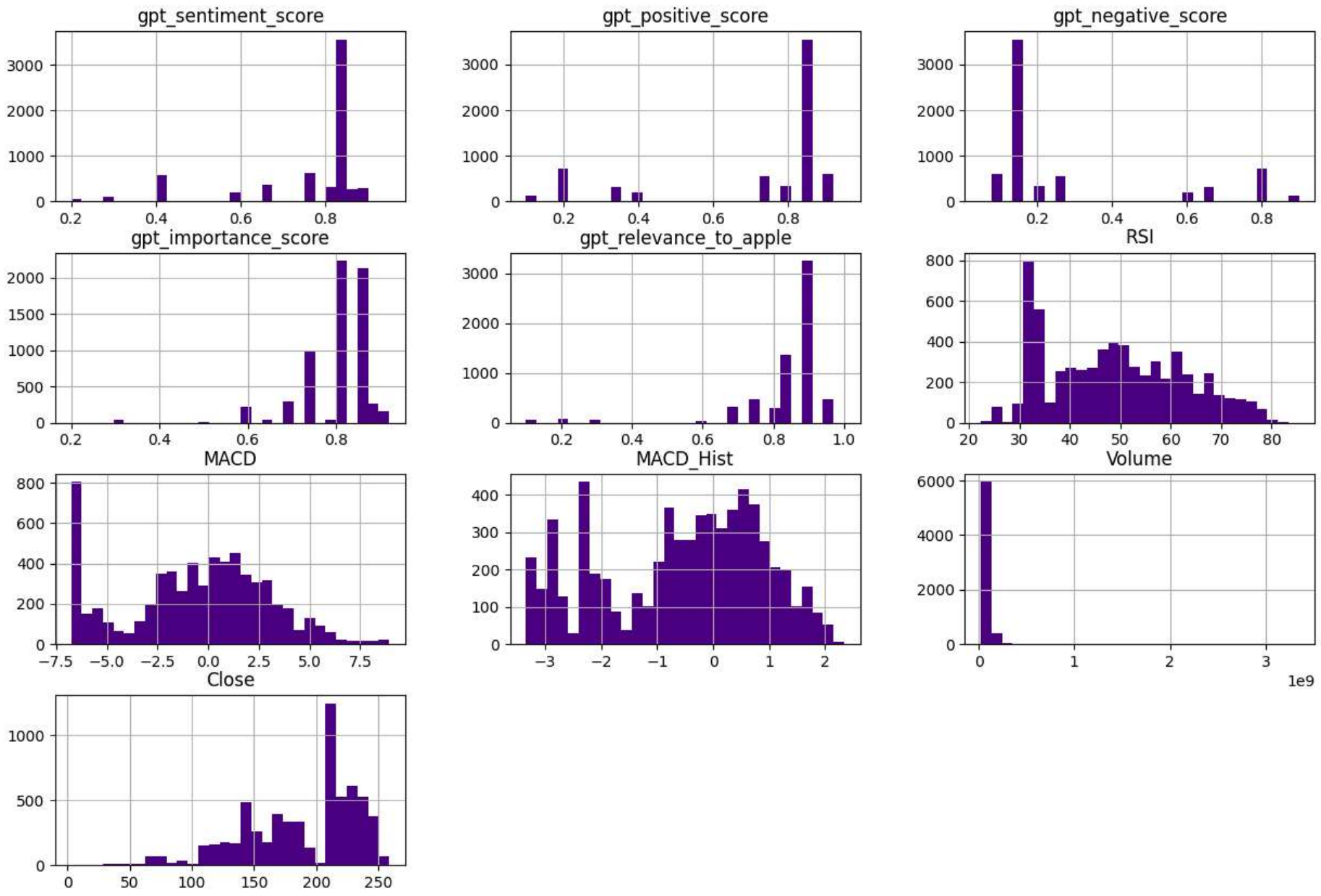


In [144…

```
df[[
    "gpt_sentiment_score", "gpt_positive_score", "gpt_negative_score",
    "gpt_importance_score", "gpt_relevance_to_apple",
    "RSI", "MACD", "MACD_Hist", "Volume", "Close"
]].hist(figsize=(15, 10), bins=30, color='indigo')
```

Out[144…

```
array([[<Axes: title={'center': 'gpt_sentiment_score'}>,
        <Axes: title={'center': 'gpt_positive_score'}>,
        <Axes: title={'center': 'gpt_negative_score'}>],
       [<Axes: title={'center': 'gpt_importance_score'}>,
        <Axes: title={'center': 'gpt_relevance_to_apple'}>,
        <Axes: title={'center': 'RSI'}>],
       [<Axes: title={'center': 'MACD'}>,
        <Axes: title={'center': 'MACD_Hist'}>,
        <Axes: title={'center': 'Volume'}>],
       [<Axes: title={'center': 'Close'}>, <Axes: >, <Axes: >]],
      dtype=object)
```



In [145…
```
df.describe()
```

Out[145…

	Open	High	Low	Close	Volume	Dividends	Stock Splits	SMA_50	SMA_200	EMA_5(
count	6415.000000	6415.000000	6415.000000	6415.000000	6.415000e+03	6415.000000	6415.000000	6415.000000	6415.000000	6415.000000
mean	189.341335	191.425654	187.196573	189.307935	7.301673e+07	0.002464	0.001871	191.548327	179.457543	190.98939
std	45.133448	45.467503	44.586510	44.955826	6.030155e+07	0.024220	0.086488	48.175300	46.602038	47.80856
min	2.601592	2.798103	2.562470	2.785764	2.323470e+07	0.000000	0.000000	2.575001	2.139975	2.53614(
25%	154.545811	155.747738	152.036458	153.912216	4.801330e+07	0.000000	0.000000	154.175806	150.785042	152.52717
50%	211.250000	213.949997	209.580002	212.690002	5.937500e+07	0.000000	0.000000	212.253434	188.762177	211.93163
75%	223.809998	225.839996	221.994983	223.983734	8.222550e+07	0.000000	0.000000	233.455149	223.910876	232.60493
max	257.906429	259.814335	257.347047	258.735504	3.349298e+09	0.250000	4.000000	240.485810	227.948397	240.88298

8 rows × 25 columns

In [146…
```
df_missing = (df.isnull().mean() * 100).round(2)
print(df_missing)
```

```
Title                     0.0
Date                      0.0
Link                      0.0
Source                    0.0
Open                      0.0
High                      0.0
Low                       0.0
Close                     0.0
Volume                    0.0
Dividends                 0.0
Stock Splits              0.0
SMA_50                    0.0
SMA_200                   0.0
EMA_50                    0.0
EMA_200                   0.0
RSI                       0.0
MACD                      0.0
MACD_Signal               0.0
MACD_Hist                 0.0
BB_Upper                  0.0
BB_Middle                 0.0
BB_Lower                  0.0
ATR                       0.0
ADX                       0.0
gpt_summary               0.0
gpt_sentiment_direction   0.0
gpt_sentiment_score       0.0
gpt_positive_score        0.0
gpt_negative_score        0.0
gpt_event_type            0.0
gpt_relevance_to_apple    0.0
gpt_importance_score      0.0
dtype: float64
```

In [147…

```
print("Duplicates:", df.duplicated().sum())
duplicates = df[df.duplicated()]
print(f"Total Duplicates: {len(duplicates)}")
duplicates
```

```
Duplicates: 2
Total Duplicates: 2
```

Out[147…

	Title	Date	Link	Source	Open	High	Low	Close	V
414	Introducing the next generation of Mac - Apple...	2020-11-10 08:00:00+00:00	https://news.google.com/rss/articles/CBMiiAFBV...	Google RSS	112.867044	114.859671	111.480009	113.27729	138
418	Introducing the next generation of Mac - Apple...	2020-11-10 08:00:00+00:00	https://news.google.com/rss/articles/CBMiiAFBV...	Google RSS	112.867044	114.859671	111.480009	113.27729	138

2 rows × 32 columns

In [148…

```
df=df.drop_duplicates()
```

In [149…

```
print("Duplicates:", df.duplicated().sum())
duplicates = df[df.duplicated()]
print(f"Total Duplicates: {len(duplicates)}")
duplicates
```

```
Duplicates: 0
Total Duplicates: 0
```

Out[149…

	Title	Date	Link	Source	Open	High	Low	Close	Volume	Dividends	…	ATR	ADX	gpt_summary	gpt_sentiment_direction	gpt_sen

0 rows × 32 columns

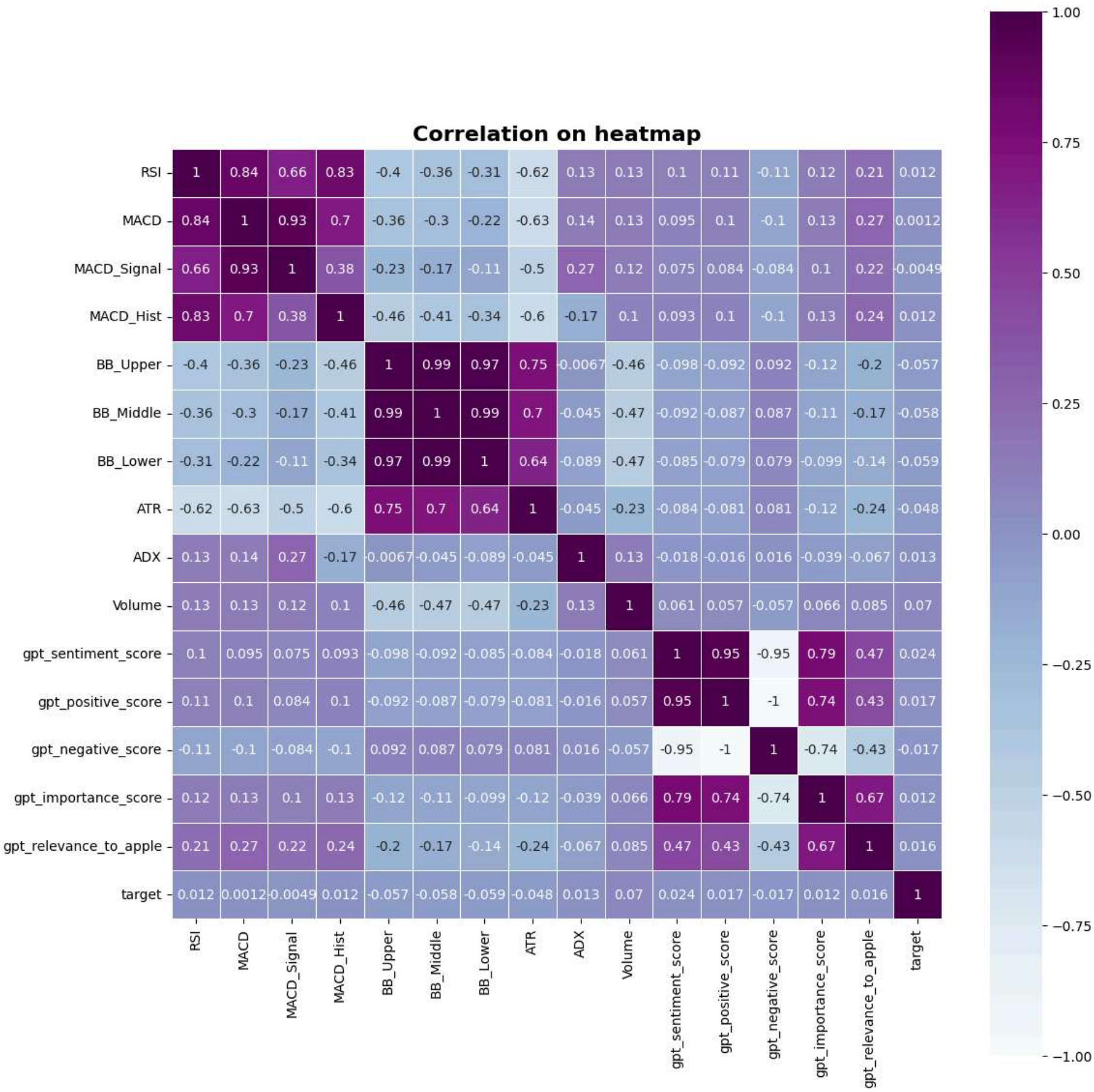
In [150…

```
df["target"] = df["Close"].shift(-1) - df["Close"]
df = df.dropna(subset=["target"])
print(df.columns)
```

```
Index(['Title', 'Date', 'Link', 'Source', 'Open', 'High', 'Low', 'Close',
       'Volume', 'Dividends', 'Stock Splits', 'SMA_50', 'SMA_200', 'EMA_50',
       'EMA_200', 'RSI', 'MACD', 'MACD_Signal', 'MACD_Hist', 'BB_Upper',
       'BB_Middle', 'BB_Lower', 'ATR', 'ADX', 'gpt_summary',
       'gpt_sentiment_direction', 'gpt_sentiment_score', 'gpt_positive_score',
       'gpt_negative_score', 'gpt_event_type', 'gpt_relevance_to_apple',
       'gpt_importance_score', 'target'],
      dtype='object')
```

In [151…
```
features=[
    "RSI", "MACD", "MACD_Signal", "MACD_Hist",
    "BB_Upper", "BB_Middle", "BB_Lower",
    "ATR", "ADX", "Volume",
    "gpt_sentiment_score", "gpt_positive_score", "gpt_negative_score",
    "gpt_importance_score", "gpt_relevance_to_apple",
]
```

In [152…
```
feature_selections= features + ["target"]
cor1=df[feature_selections].corr()
plt.figure(figsize=(12,12))
sb.heatmap(cor1, annot=True, cmap='BuPu', linewidths=0.5, square=True)
plt.title('Correlation on heatmap', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```

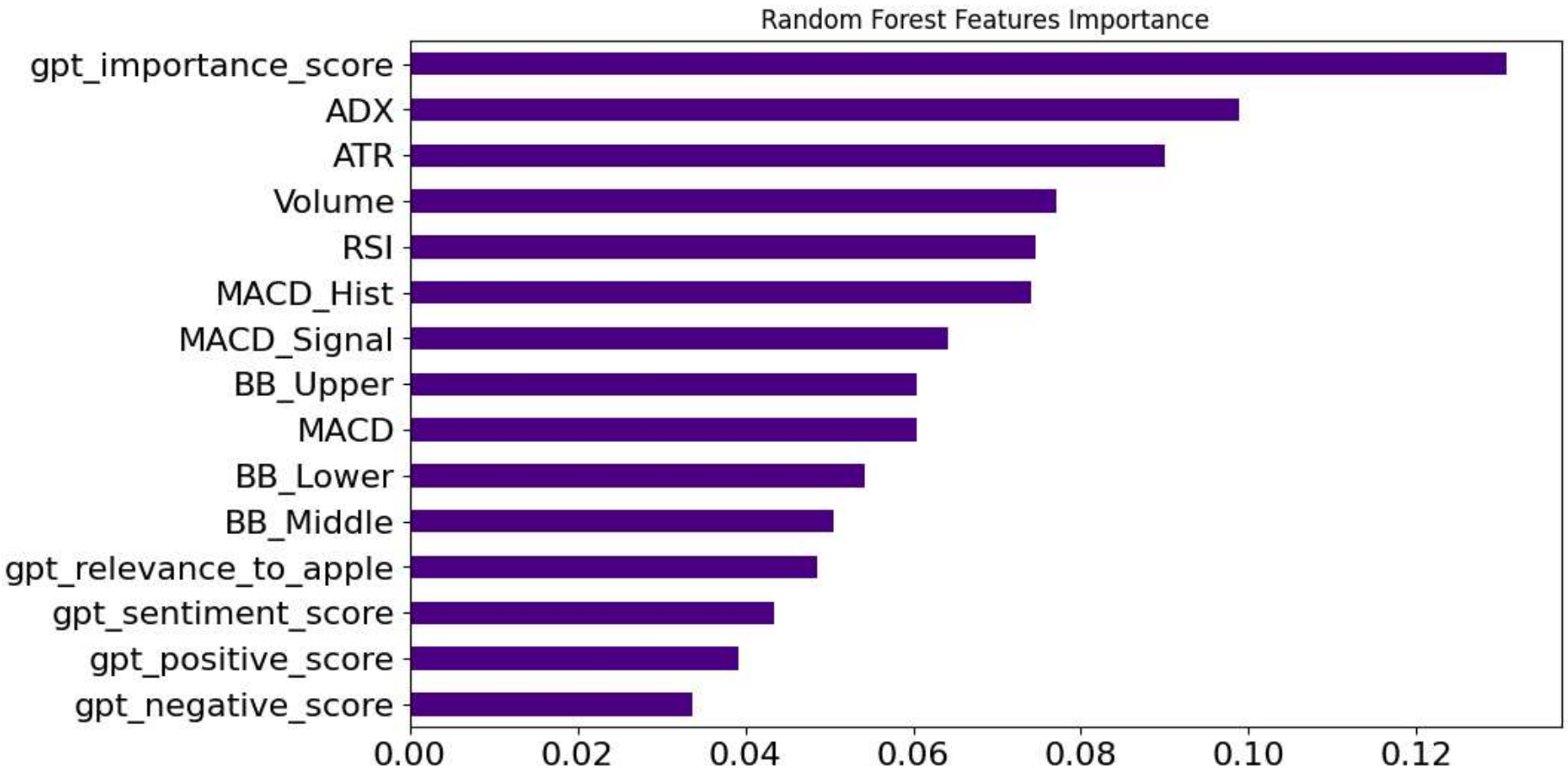


Features of Importance

In [153…
```
x= df[features]
y= df['target']
x_train, x_test, y_train, y_test= train_test_split(x,y,test_size=0.3, random_state=42)
print("X_train: ", x.shape)
print("Y_train: ", y.shape)

rf_imp= RandomForestRegressor(n_estimators=50, random_state=42)
rf_imp.fit(x_train, y_train)

importances = rf_imp.feature_importances_
importances_df = pd.Series(importances, index=features).sort_values()
```

```
importances_df.plot (kind='barh', color='indigo', title='Random Forest Features Importance', fontsize='16', figsize=(10,6))
print("\n Scores of Importances\n", importances)
```

```
X_train:  (6412, 15)
Y_train:  (6412,)

 Scores of Importances
 [0.07453338 0.06038833 0.06413385 0.07413538 0.06048859 0.05056873
 0.05415478 0.08993331 0.09895428 0.07709433 0.0435145  0.03909295
 0.03375041 0.13075121 0.04850598]
```



We applied a Random Forest Regressor to assess the relative importance of both technical indicators and GPT-enriched news features in predicting next-day Apple stock price movement.

Contrary to initial assumptions, the feature *gpt_importance_score* emerged as the most influential predictor, even surpassing traditional indicators like *Volume, RSI, and MACD*.

This finding highlights the growing value of semantic and contextual understanding from financial news. Other significant contributors included ADX, ATR, and MACD_Hist, indicating that combining sentiment enrichment with volatility and momentum indicators offers a more holistic model perspective.

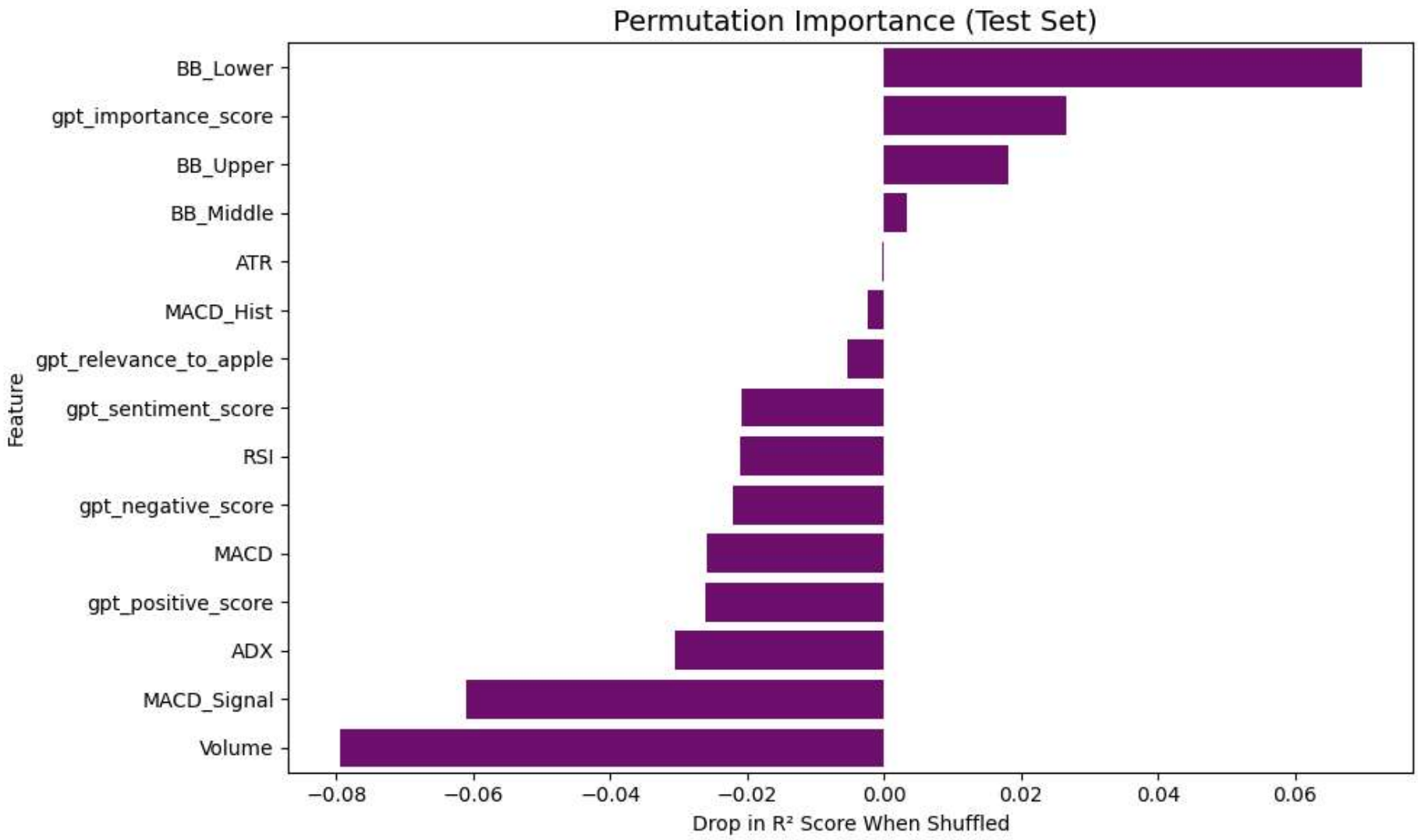
Permutation Importance

In [154…

```
result = permutation_importance(rf_imp, x_test,y_test,n_repeats=10,random_state=42,scoring='r2')
perm_df = pd.DataFrame({
    "Feature": x_test.columns,
    "Importance Mean": result.importances_mean,
    "Importance Std": result.importances_std
}).sort_values(by="Importance Mean", ascending=False)


print("\n Permutation Importances:\n")
print(perm_df.to_string(index=False))

plt.figure(figsize=(10, 6))
sb.barplot(x="Importance Mean", y="Feature", data=perm_df, color='purple')
plt.title(" Permutation Importance (Test Set)", fontsize=14)
plt.xlabel("Drop in R² Score When Shuffled")
plt.tight_layout()
plt.show()
```

Permutation Importances:

Feature	Importance Mean	Importance Std
BB_Lower	0.069679	0.019818
gpt_importance_score	0.026496	0.013914
BB_Upper	0.018079	0.014028
BB_Middle	0.003263	0.012690
ATR	-0.000219	0.012628
MACD_Hist	-0.002350	0.015113
gpt_relevance_to_apple	-0.005431	0.005436
gpt_sentiment_score	-0.020754	0.009595
RSI	-0.020955	0.007384
gpt_negative_score	-0.022051	0.003971
MACD	-0.025888	0.013345
gpt_positive_score	-0.026136	0.005060
ADX	-0.030596	0.021532
MACD_Signal	-0.061010	0.013115
Volume	-0.079485	0.011329



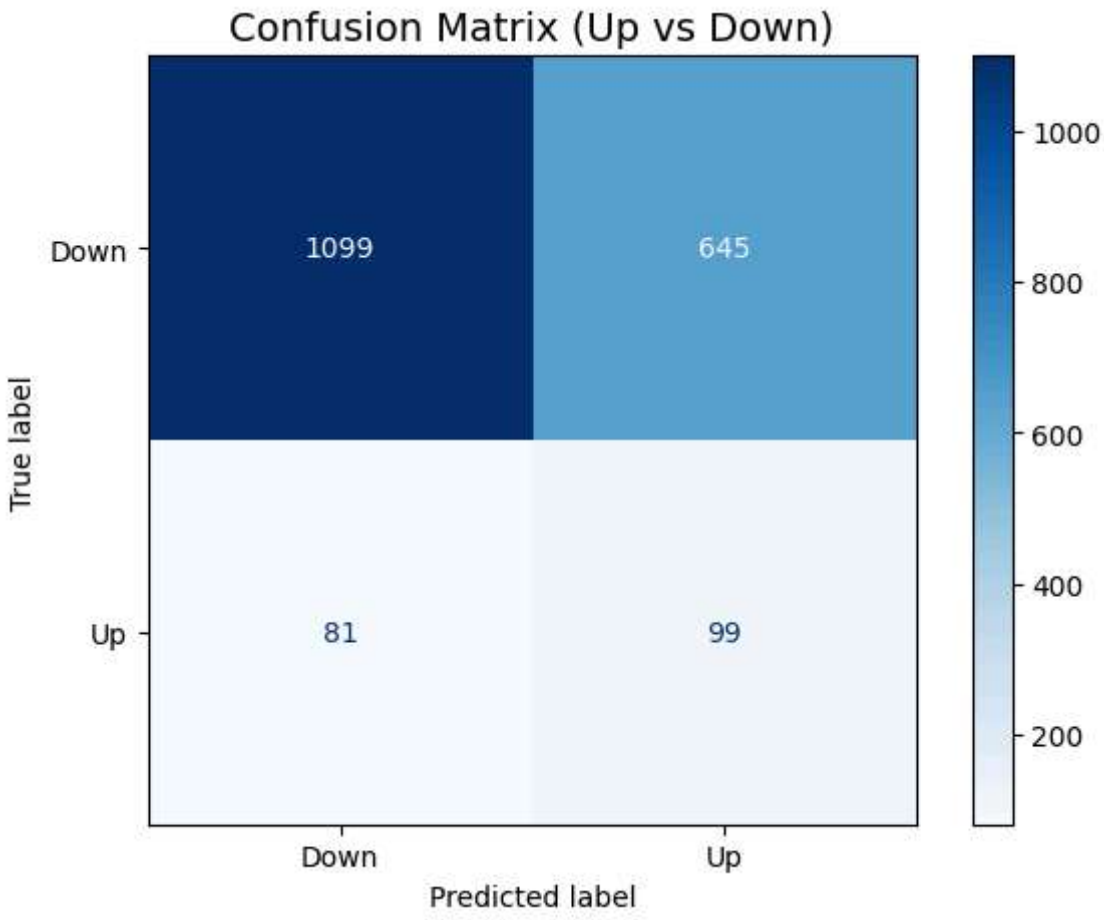
To validate model reliance on individual features, we applied permutation importance on the test dataset. The analysis revealed that BB_Lower and gpt_importance_score had the most significant influence on prediction accuracy, as shuffling these features led to the largest drop in R^2 score. This reinforces the value of combining lower Bollinger Band technical indicators with GPT-derived relevance scoring. Interestingly, features like Volume and MACD_Signal showed a negative impact on model performance, suggesting potential redundancy or overfitting on those signals. Overall, the results confirm that enriched sentiment attributes and select technical indicators complement each other in enhancing model robustness.

In [166…

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

x= df[features]
y= df['target']
x_train, x_test, y_train, y_test= train_test_split(x,y,test_size=0.3, random_state=42)
print("X_train: ", x.shape)
print("Y_train: ", y.shape)

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestRegressor(n_estimators=100, random_state=42))
])

pipeline.fit(x_train, y_train)

y_pred = pipeline.predict(x_test)
print("R² Score:", r2_score(y_test, y_pred))
print("Mean Absolute Error:", mean_absolute_error(y_test, y_pred))
print("Root Mean Squared Error:", np.sqrt(mean_squared_error(y_test, y_pred)))

# Convert continuous predictions to classes (Up/Down)
def label_class(val):
    return 1 if val > 0 else 0  # 1 = price up, 0 = price down/stable

y_test_class = y_test.apply(label_class)
y_pred_class = pd.Series(y_pred).apply(label_class)
```

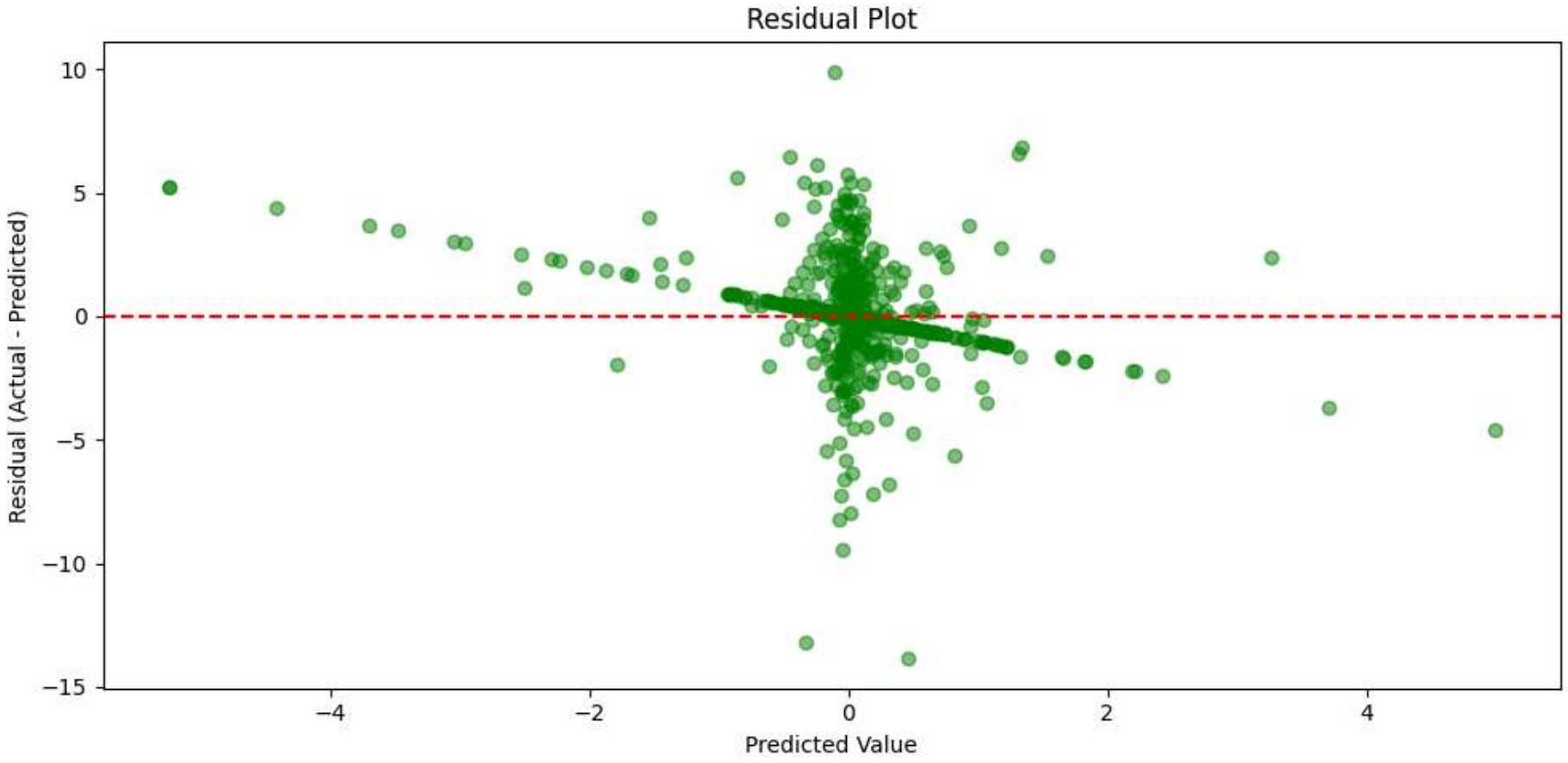
```
# Accuracy & Confusion Matrix
print("Accuracy Score:", accuracy_score(y_test_class, y_pred_class))

cm = confusion_matrix(y_test_class, y_pred_class)
print("\nconfusion matrix", cm)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Down", "Up"])
disp.plot(cmap='Blues')

plt.title("Confusion Matrix (Up vs Down)", fontsize=14)
plt.tight_layout()
plt.show()
```

```
X_train:  (6412, 15)
Y_train:  (6412,)
R² Score: -0.2257432126985901
Mean Absolute Error: 0.5724596228981614
Root Mean Squared Error: 1.3217709140728455
Accuracy Score: 0.6226611226611226

confusion matrix [[1099  645]
 [  81   99]]
```



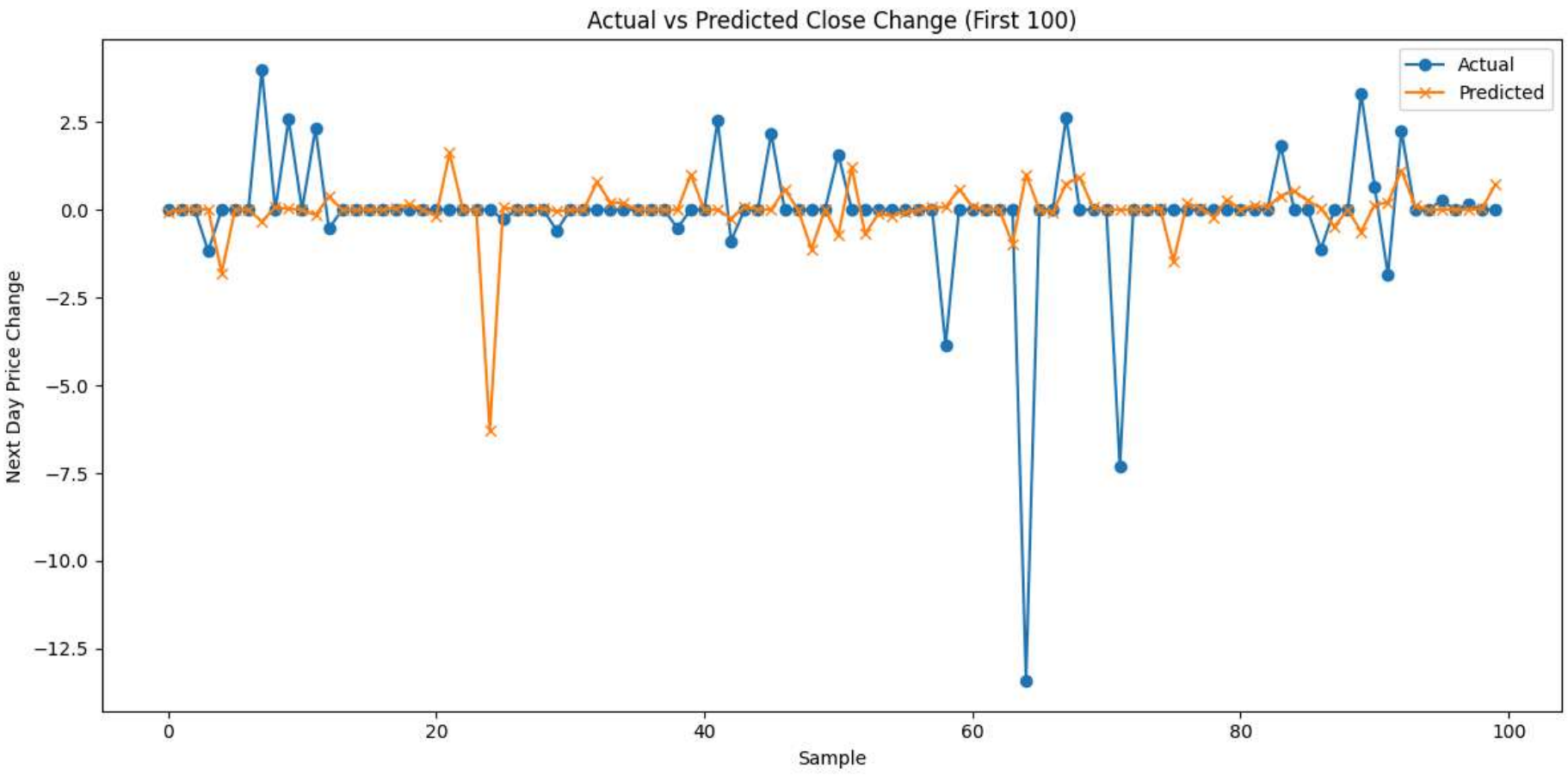
In [184…

```
residuals = y_test - y_pred

plt.figure(figsize=(10,5))
plt.scatter(y_pred, residuals, alpha=0.5, color='green')
plt.axhline(0, color='red', linestyle='--')
plt.title("Residual Plot")
plt.xlabel("Predicted Value")
plt.ylabel("Residual (Actual - Predicted)")
plt.tight_layout()
plt.show()
```



In [157…

```
plt.figure(figsize=(12,6))
plt.plot(y_test.values[:100], label="Actual", marker='o')
plt.plot(y_pred[:100], label="Predicted", marker='x')
plt.title("Actual vs Predicted Close Change (First 100)")
plt.xlabel("Sample")
plt.ylabel("Next Day Price Change")
plt.legend()
plt.tight_layout()
plt.show()
```

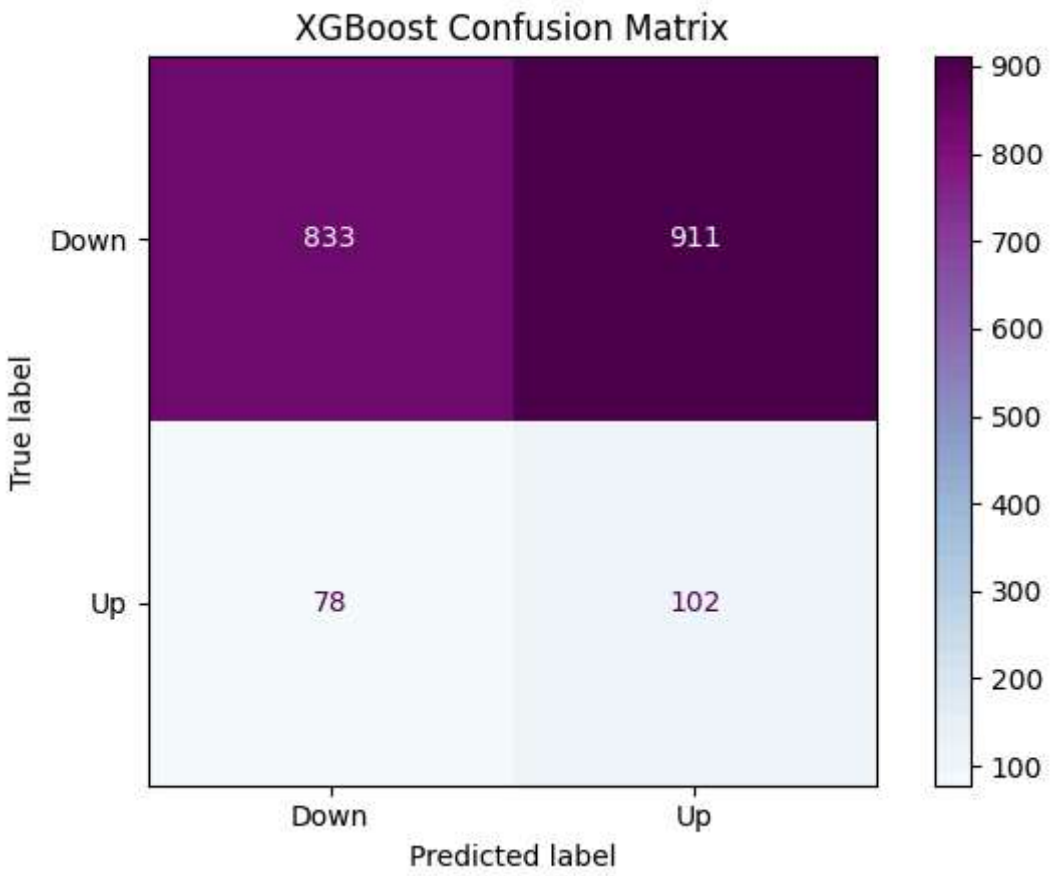


In [187…

```
import xgboost as xgb

xgb_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('xgb', xgb.XGBRegressor(
        n_estimators=50,
        learning_rate=0.2,
        max_depth=10,
        random_state=42,
        objective='reg:squarederror'
    ))
])

xgb_pipeline.fit(x_train, y_train)
y_pred_XGB = xgb_pipeline.predict(x_test)

print("\nXGBoost Regressor Evaluation:")
print("R² Score:", r2_score(y_test, y_pred_XGB))
print("MAE:", mean_absolute_error(y_test, y_pred_XGB))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_XGB)))

# === Classification Metrics ===
```

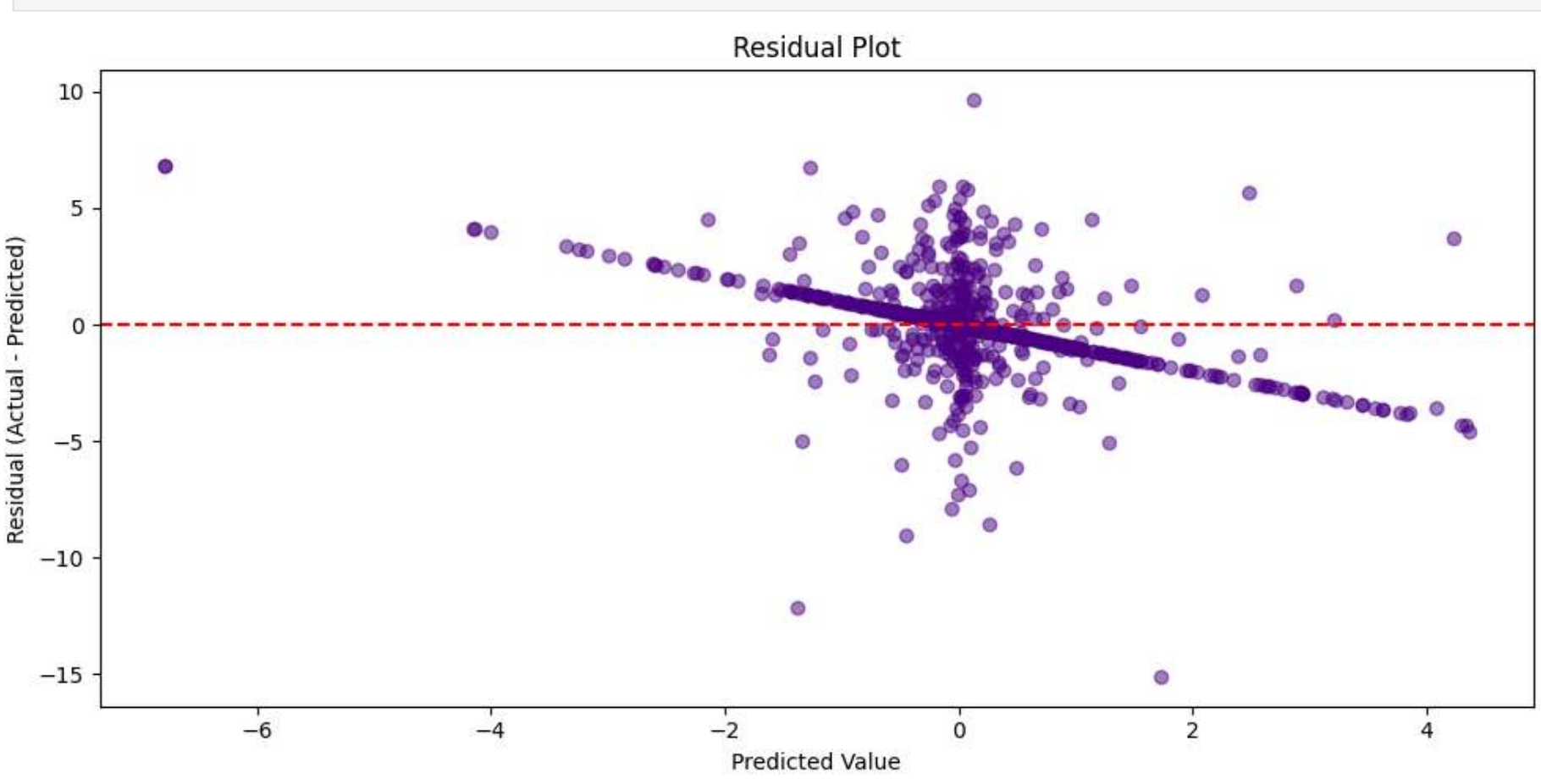
```python
def label_class(val):
    return 1 if val > 0 else 0

y_test_class = y_test.apply(label_class)
y_pred_class = pd.Series(y_pred_XGB).apply(label_class)

print("Accuracy Score:", accuracy_score(y_test_class, y_pred_class))

cm = confusion_matrix(y_test_class, y_pred_class)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Down", "Up"])
disp.plot(cmap='BuPu')
plt.title("\nXGBoost Confusion Matrix")
plt.tight_layout()
plt.show()
```

```
XGBoost Regressor Evaluation:
R² Score: -0.2630948249714471
MAE: 0.589031344323393
RMSE: 1.3417587008310004
Accuracy Score: 0.48596673596673595
```



In [193…

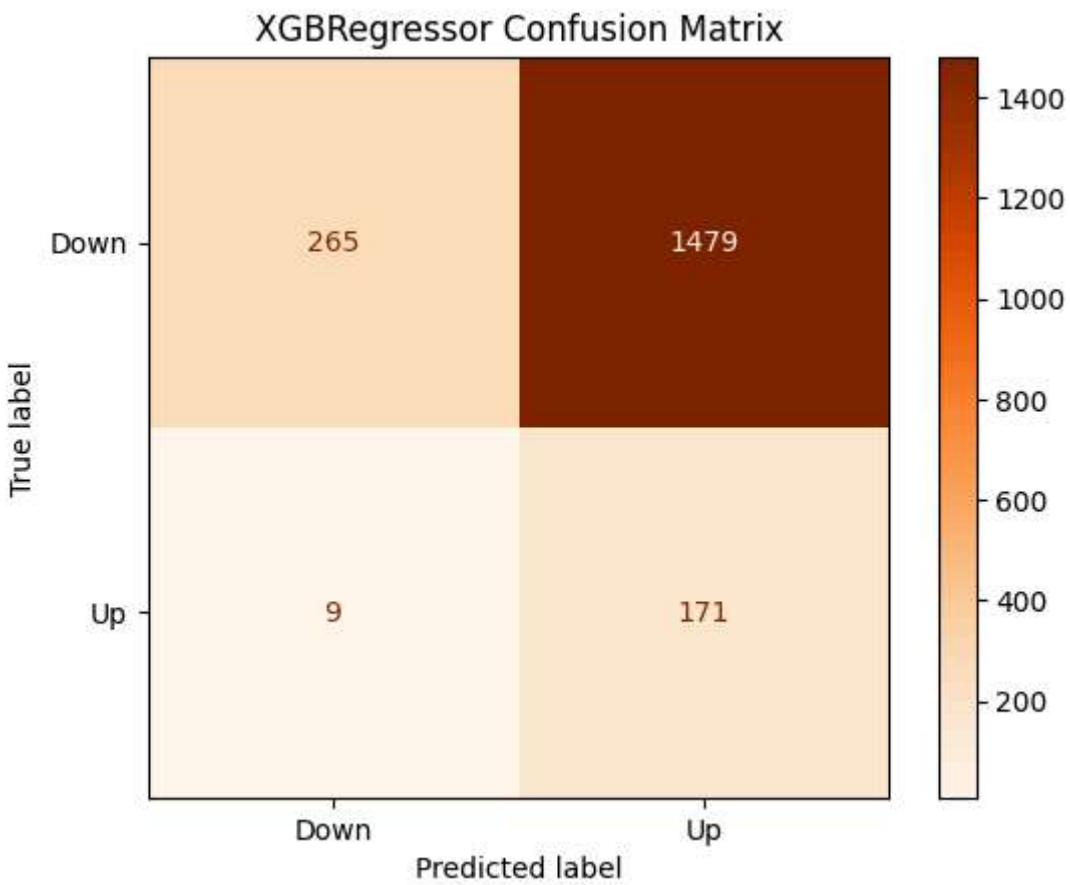
```python
residuals = y_test - y_pred_XGB

plt.figure(figsize=(10,5))
plt.scatter(y_pred_XGB, residuals, alpha=0.5, color='Indigo')
plt.axhline(0, color='red', linestyle='--')
plt.title("Residual Plot")
plt.xlabel("Predicted Value")
plt.ylabel("Residual (Actual - Predicted)")
plt.tight_layout()
plt.show()
```

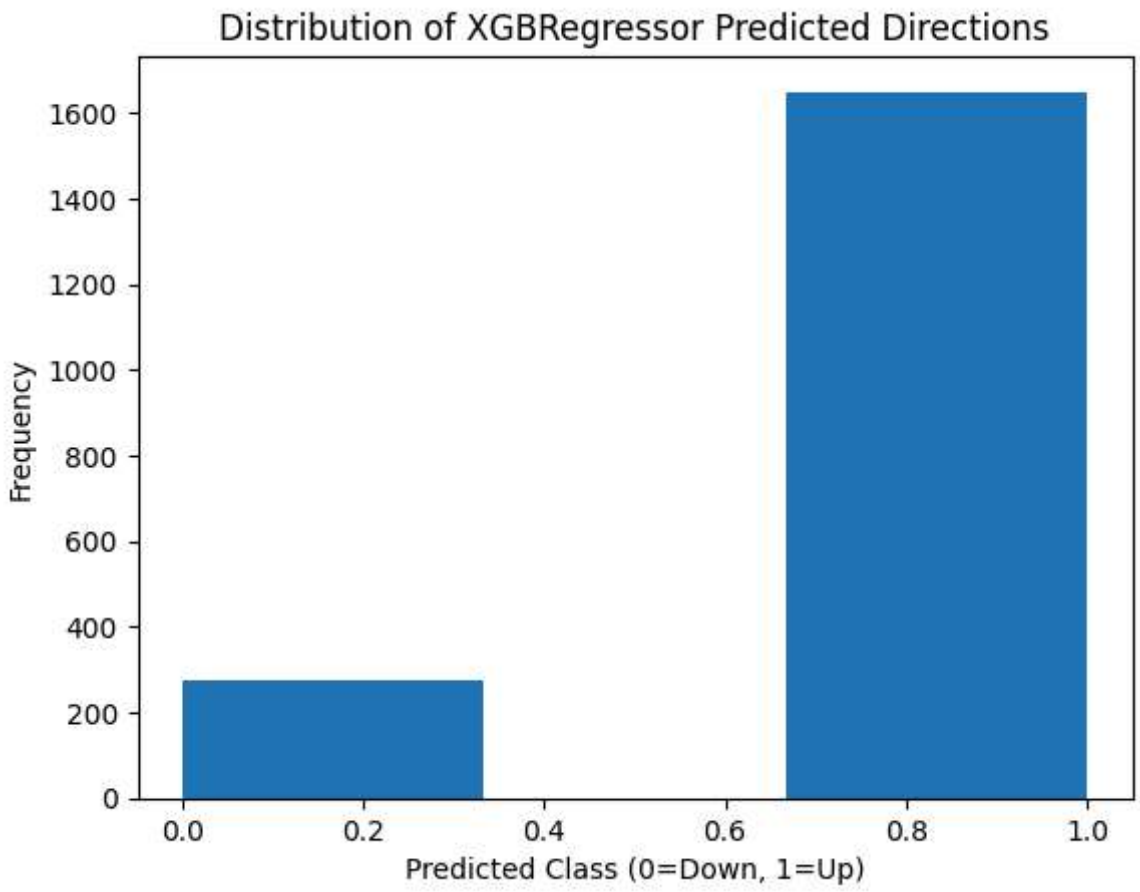


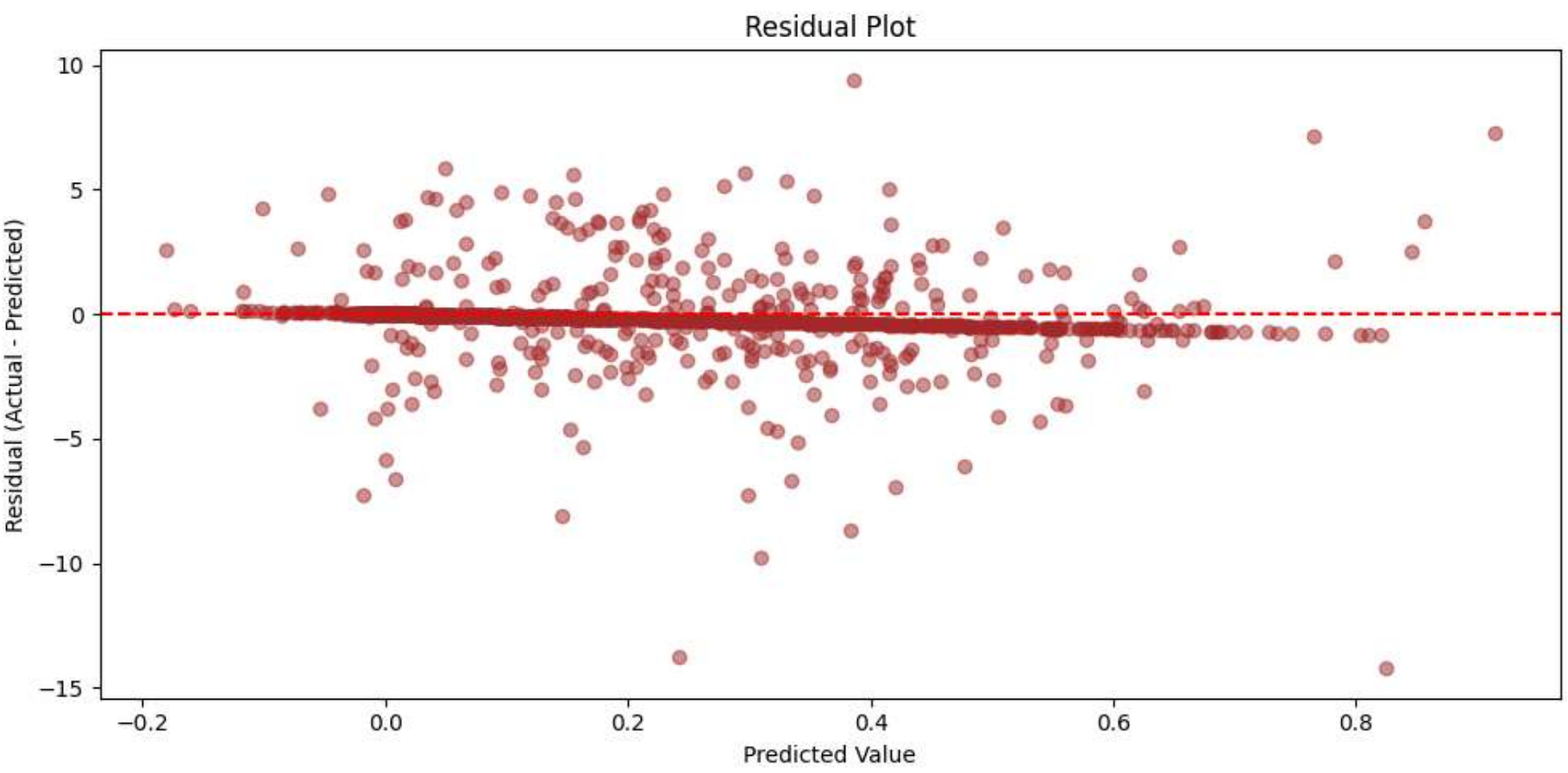
In [203…

```python
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.pipeline import Pipeline


def label_class(val): return 1 if val > 0 else 0
y_class = y.apply(label_class)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
y_train_class = y_train.apply(label_class)
y_test_class = y_test.apply(label_class)

smote = SMOTE(random_state=42)
x_train_resampled, y_train_class_resampled = smote.fit_resample(x_train, y_train_class)

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('xgb', XGBRegressor(n_estimators=100, learning_rate=0.1, random_state=42))
])

pipeline.fit(x_train_resampled, y_train_class_resampled)
y_pred_gbr = pipeline.predict(x_test)
print(" Regression Performance:")
print("R² Score:", r2_score(y_test, y_pred_gbr))
print("MAE:", mean_absolute_error(y_test, y_pred_gbr))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_gbr)))

y_pred_class = pd.Series(y_pred_gbr).apply(label_class)
print("\n Classification Accuracy:", accuracy_score(y_test_class, y_pred_class))

cm = confusion_matrix(y_test_class, y_pred_class)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Down", "Up"])
disp.plot(cmap='Oranges')
plt.title("XGBRegressor Confusion Matrix")
plt.show()

plt.hist(y_pred_class, bins=3)
plt.title("Distribution of XGBRegressor Predicted Directions")
plt.xlabel("Predicted Class (0=Down, 1=Up)")
plt.ylabel("Frequency")
plt.show()

residuals = y_test - y_pred_gbr

plt.figure(figsize=(10,5))
plt.scatter(y_pred_gbr, residuals, alpha=0.5, color='brown')
plt.axhline(0, color='red', linestyle='--')
plt.title("Residual Plot")
plt.xlabel("Predicted Value")
plt.ylabel("Residual (Actual - Predicted)")
plt.tight_layout()
plt.show()
```

```
 Regression Performance:
R² Score: -0.027775066701894113
MAE: 0.47734131620364645
RMSE: 1.2103346905012795

 Classification Accuracy: 0.22661122661122662
```







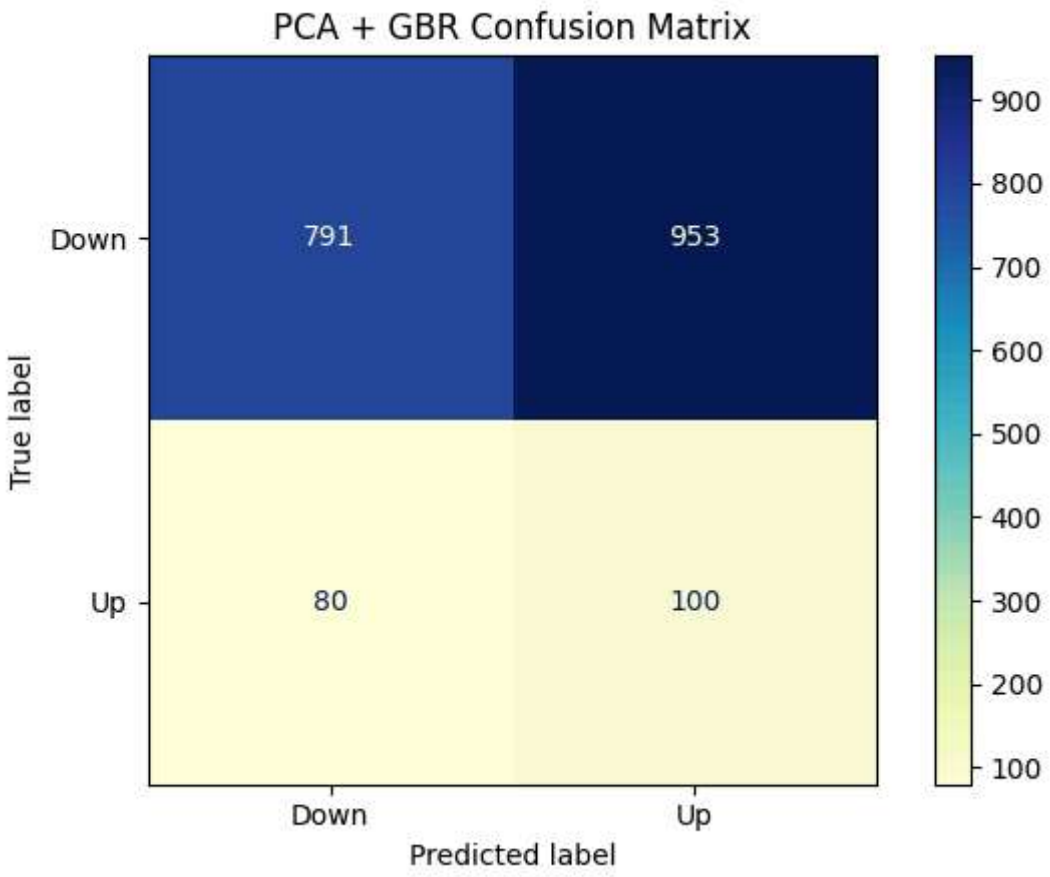
In [ ]:

In [197…

```
from sklearn.decomposition import PCA
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.pipeline import Pipeline

# Replace scaler with PCA
pca_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=0.95)),
    ('gbr', GradientBoostingRegressor(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=5,
        random_state=42
    ))
])

# Train
pca_pipeline.fit(x_train, y_train)

# Predict
y_pred_GBR = pca_pipeline.predict(x_test)

# Evaluate
print("\nPCA + GradientBoostingRegressor Evaluation:")
print("R² Score:", r2_score(y_test, y_pred_GBR))
print("MAE:", mean_absolute_error(y_test, y_pred_GBR))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_GBR)))

# Classification-like Up/Down
y_test_class = y_test.apply(lambda v: 1 if v > 0 else 0)
```

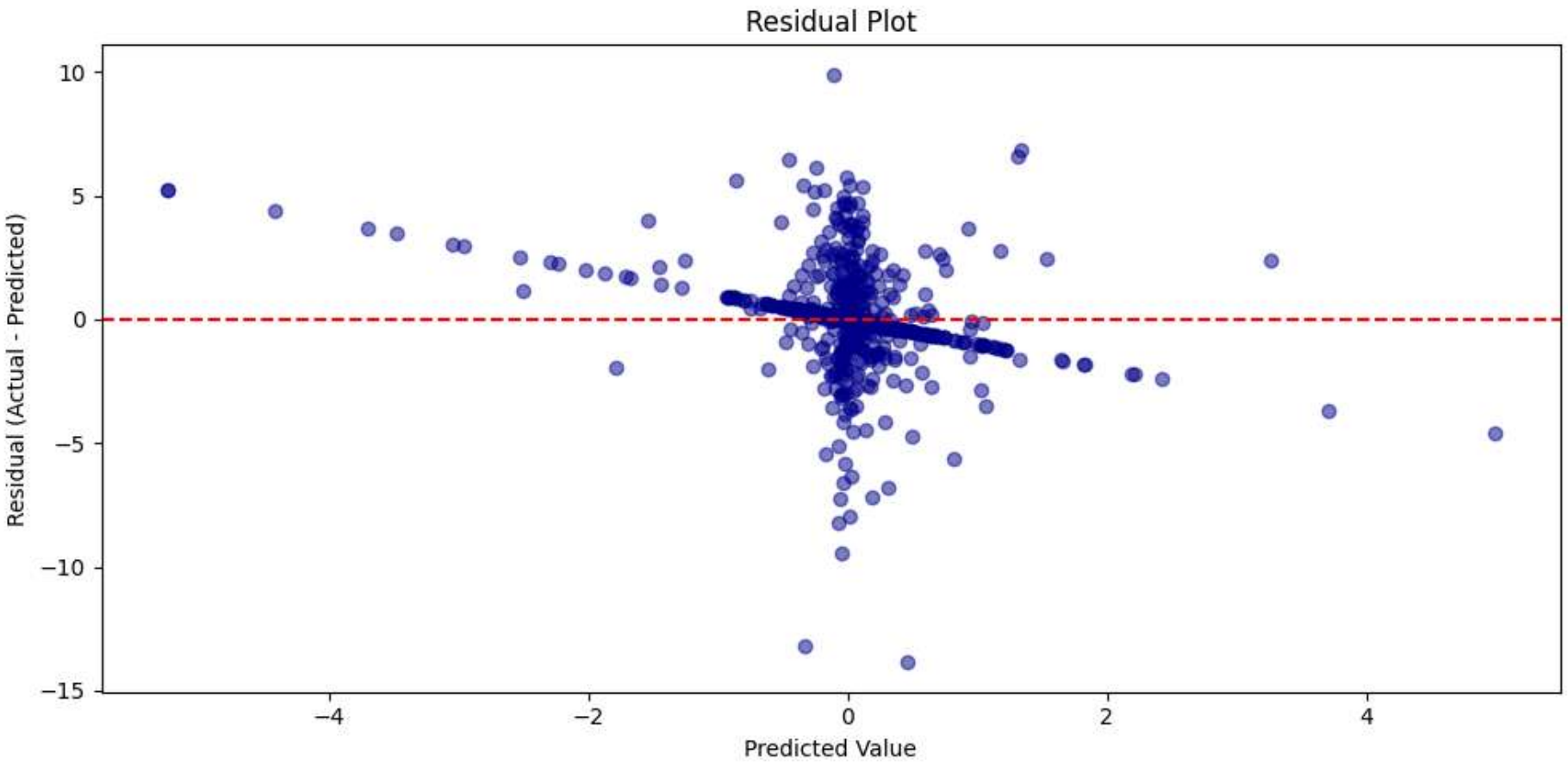
```
y_pred_class = pd.Series(y_pred_GBR).apply(lambda v: 1 if v > 0 else 0)

print("Accuracy Score:", accuracy_score(y_test_class, y_pred_class))

cm = confusion_matrix(y_test_class, y_pred_class)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Down", "Up"])
disp.plot(cmap='YlGnBu')
plt.title("\nPCA + GBR Confusion Matrix")
plt.tight_layout()
plt.show()
```

```
PCA + GradientBoostingRegressor Evaluation:
R² Score: -0.09278728628318711
MAE: 0.48483141119656237
RMSE: 1.248027799246534
Accuracy Score: 0.4630977130977131
```



In [200…

```
residuals = y_test - y_pred_GBR

plt.figure(figsize=(10,5))
plt.scatter(y_pred_GBR, residuals, alpha=0.5, color='darkblue')
plt.axhline(0, color='red', linestyle='--')
plt.title("Residual Plot")
plt.xlabel("Predicted Value")
plt.ylabel("Residual (Actual - Predicted)")
plt.tight_layout()
plt.show()
```

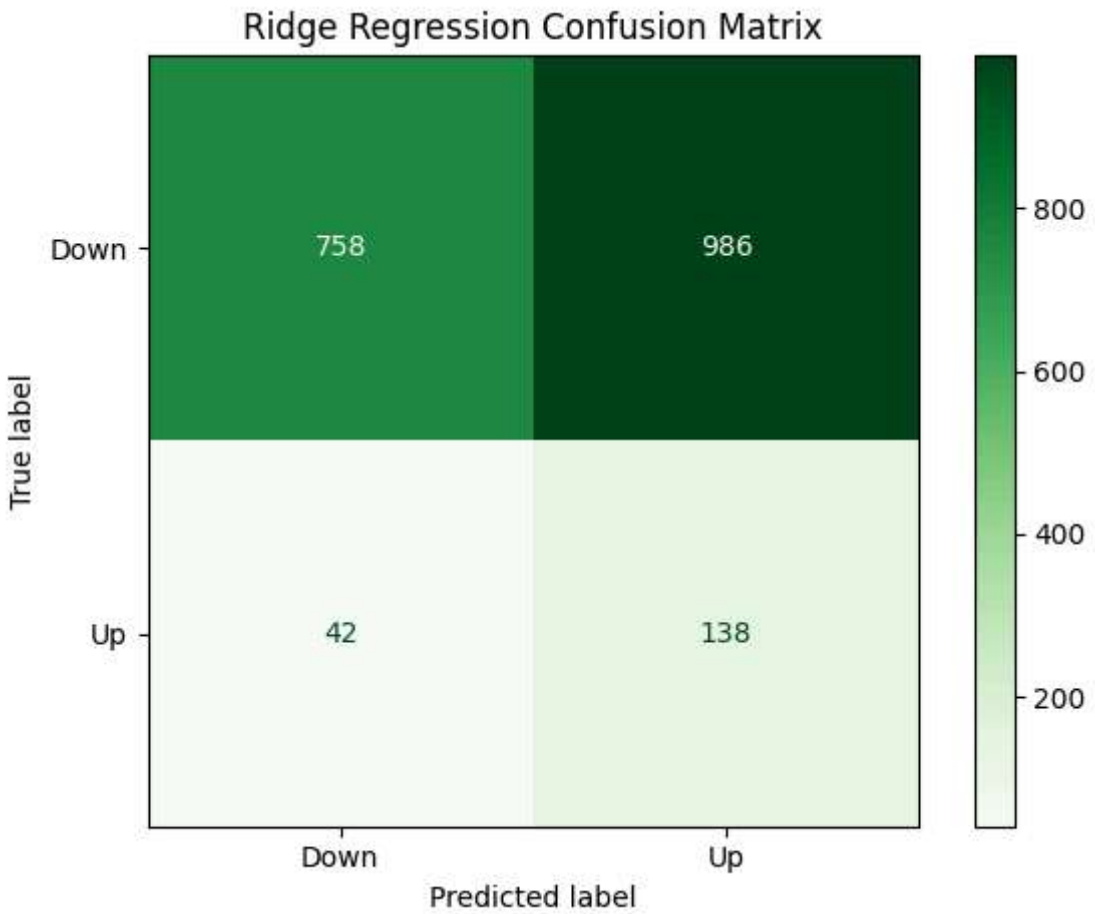


In [179…

```
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
ridge_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('ridge', Ridge(alpha=1.0))  # You can tune alpha
])

ridge_pipeline.fit(x_train, y_train)
y_pred = ridge_pipeline.predict(x_test)

print("\nRidge Regressor Evaluation:")
print("R² Score:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))

def label_class(val):
    return 1 if val > 0 else 0

y_test_class = y_test.apply(label_class)
y_pred_class = pd.Series(y_pred).apply(label_class)

print("Accuracy Score:", accuracy_score(y_test_class, y_pred_class))

cm = confusion_matrix(y_test_class, y_pred_class)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Down", "Up"])
disp.plot(cmap='Greens')
plt.title("Ridge Regression Confusion Matrix")
plt.tight_layout()
plt.show()
```

```
Ridge Regressor Evaluation:
R² Score: 0.0016738066660882955
MAE: 0.42364479961414103
RMSE: 1.1928687882536455
Accuracy Score: 0.4656964656964657
```

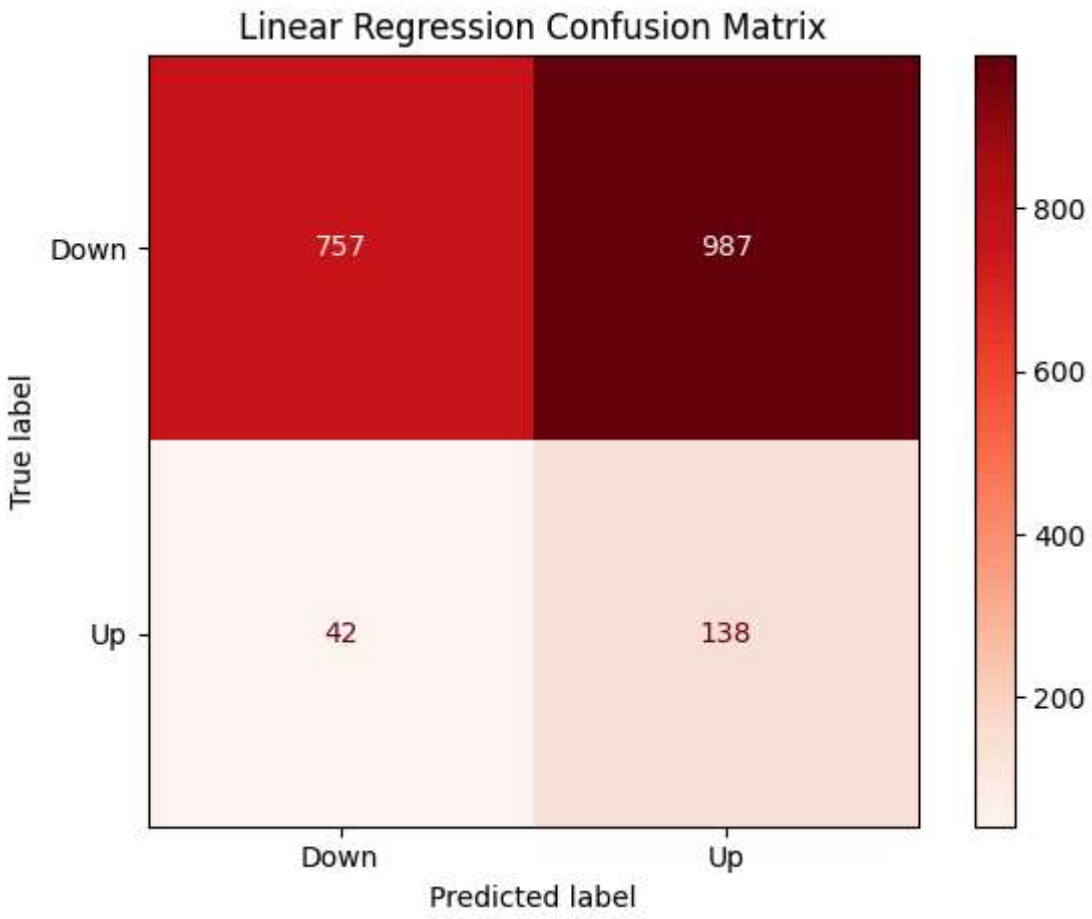


In [ ]:

```
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# === Define Pipeline ===
linear_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('linear', LinearRegression())
])

# === Train the model ===
linear_pipeline.fit(x_train, y_train)

# === Predict ===
y_pred_lig = linear_pipeline.predict(x_test)

# === Regression Evaluation ===
print("\nLinear Regression Evaluation:")
print("R² Score:", r2_score(y_test, y_pred_lig))
print("MAE:", mean_absolute_error(y_test, y_pred_lig))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lig)))

# === Classification-style Up/Down ===
def label_class(val):
    return 1 if val > 0 else 0
```

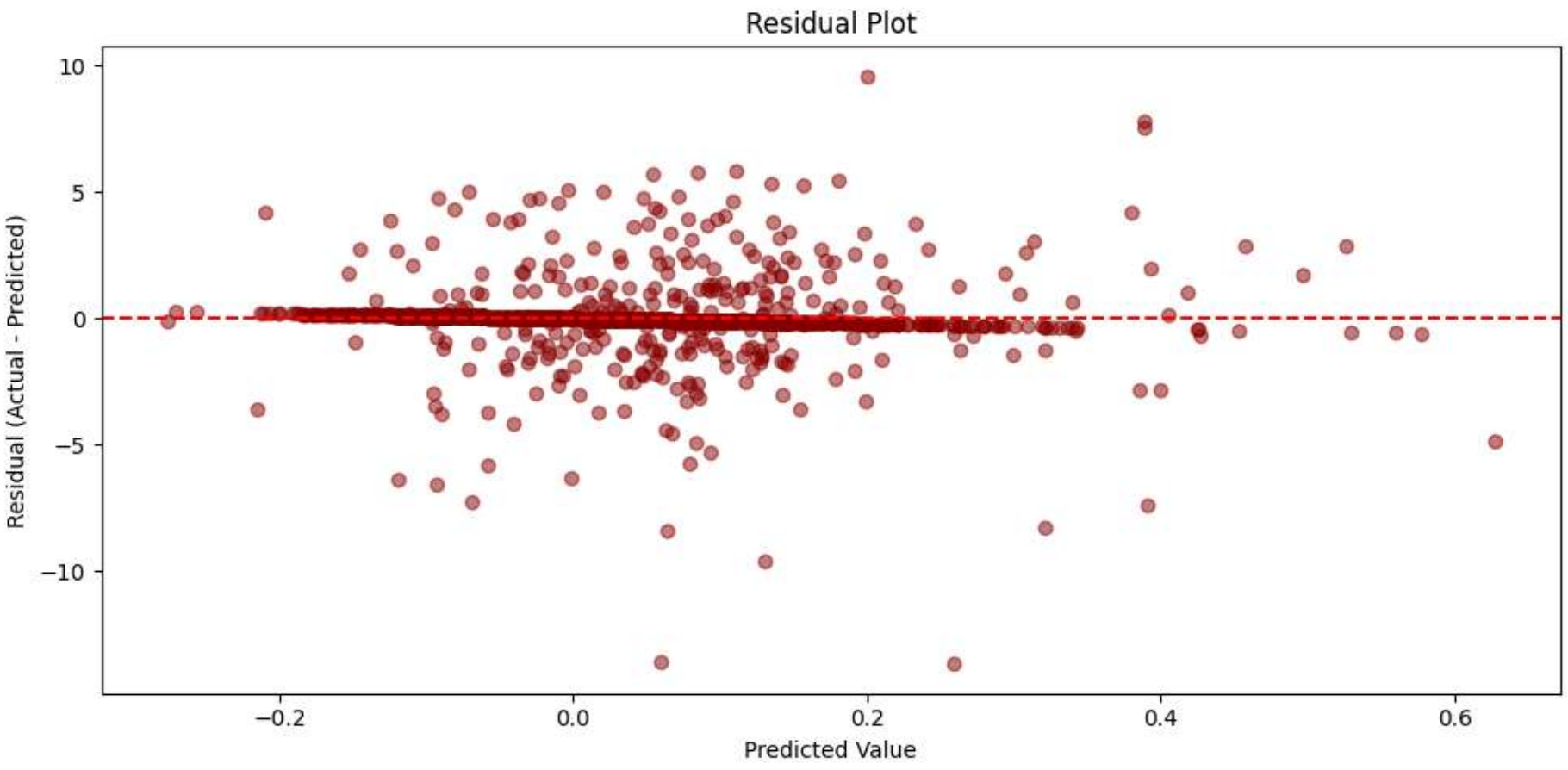
```
y_test_class = y_test.apply(label_class)
y_pred_class = pd.Series(y_pred_lig).apply(label_class)

# === Classification Metrics ===
print("Accuracy Score:", accuracy_score(y_test_class, y_pred_class))

cm = confusion_matrix(y_test_class, y_pred_class)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Down", "Up"])
disp.plot(cmap='Reds')
plt.title("Linear Regression Confusion Matrix")
plt.tight_layout()
plt.show()
```

```
Linear Regression Evaluation:
R² Score: 0.001655563475164712
MAE: 0.42379632379075766
RMSE: 1.1928796873133816
Accuracy Score: 0.46517671517671516
```



In [207…

```
residuals = y_test - y_pred_lig

plt.figure(figsize=(10,5))
plt.scatter(y_pred_lig, residuals, alpha=0.5, color='darkred')
plt.axhline(0, color='red', linestyle='--')
plt.title("Residual Plot")
plt.xlabel("Predicted Value")
plt.ylabel("Residual (Actual - Predicted)")
plt.tight_layout()
plt.show()
```



In [ ]:

```
from sklearn.preprocessing import PolynomialFeatures
```

```python
poly_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('poly', PolynomialFeatures(degree=3, include_bias=False)),
    ('regressor', LinearRegression())
])

poly_pipeline.fit(x_train, y_train)
y_pred_poly = poly_pipeline.predict(x_test)

print("\nPolynomial Regression (Degree 3) Evaluation:")
print("R² Score:", r2_score(y_test, y_pred_poly))
print("MAE:", mean_absolute_error(y_test, y_pred_poly))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_poly)))

y_test_class = y_test.apply(lambda v: 1 if v > 0 else 0)
y_pred_class = pd.Series(y_pred_poly).apply(lambda v: 1 if v > 0 else 0)

print("Accuracy Score:", accuracy_score(y_test_class, y_pred_class))

cm = confusion_matrix(y_test_class, y_pred_class)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Down", "Up"])
disp.plot(cmap='PuBuGn')
plt.title("Polynomial Regression Confusion Matrix")
plt.tight_layout()
plt.show()
```

```
Polynomial Regression (Degree 2) Evaluation:
R² Score: -0.40491404721003654
MAE: 0.6537088688098409
RMSE: 1.4150810649423566
Accuracy Score: 0.4864864864864865
```